

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Lê Minh Hoàng-25520583

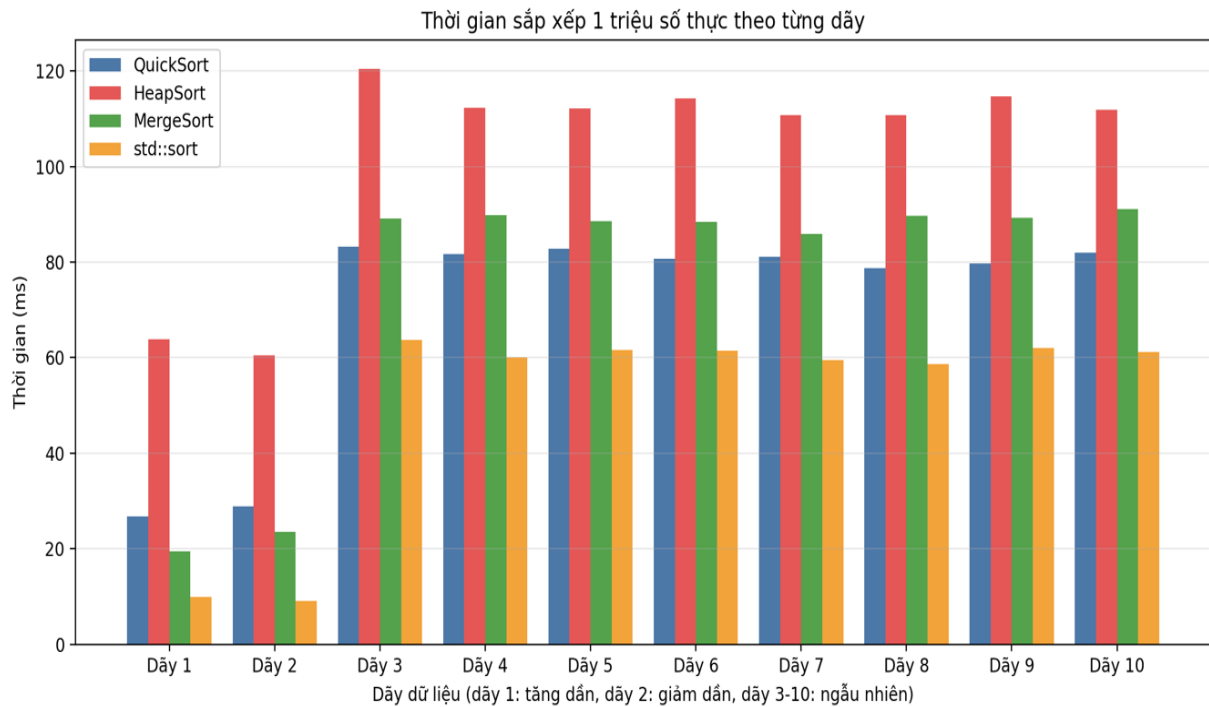
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (C++)
1	26.7404	63.917	19.4904	9.9213
2	28.9241	60.5118	23.6055	9.0138
3	83.2237	120.545	89.1703	63.6526
4	81.7354	112.398	89.8648	60.0802
5	82.858	112.156	88.5652	61.6556
6	80.7012	114.301	88.4311	61.4104
7	81.0884	110.784	85.8724	59.4597
8	78.7718	110.746	89.6594	58.6469
9	79.747	114.706	89.256	62.1004
10	82.051	111.945	91.1004	61.1328
Trung bình	Sắp xếp: 27.8 Ngẫu nhiên:81.3	Sắp xếp:62.2 Ngẫu nhiên:113.4	Sắp xếp:21.5 Ngẫu nhiên:89.0	Sắp xếp:9.5 Ngẫu nhiên:61.0

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

1. `std::sort` (C++) là nhanh nhất và ổn định nhất.

Trên dữ liệu ngẫu nhiên, `std::sort` có thời gian thấp nhất trong 4 thuật toán. Nó dùng introsort (QuickSort + HeapSort + InsertionSort), nên vừa nhanh như QuickSort vừa tránh được trường hợp xấu $O(n^2)$. Nó cũng được tối ưu kỹ bởi trình biên dịch.

2. QuickSort nhanh, gần với `std::sort`, nhưng phụ thuộc cách chọn pivot.

Độ phức tạp trung bình là $O(n \log n)$, thao tác trên mảng liên tục nên tận dụng cache tốt. Với pivot median-of-three, QuickSort xử lý tốt cả dãy tăng dần và giảm dần. Nếu chọn pivot ở đầu hoặc cuối thì hai dãy này sẽ rơi vào $O(n^2)$ và có thể tràn stack. Đây là điểm yếu cần nêu rõ trong báo cáo.

3. MergeSort ổn định về thời gian nhưng chậm hơn và tốn thêm bộ nhớ.

Luôn $O(n \log n)$ cho mọi loại dữ liệu, thời gian ít chênh lệch giữa các dãy. Tuy nhiên cần mảng phụ $O(n)$ và phải sao chép dữ liệu qua lại, nên chậm hơn QuickSort và `std::sort` trên dữ liệu ngẫu nhiên. Ưu điểm là sắp xếp ổn định (stable) và không có trường hợp xấu.

4. HeapSort chậm nhất trên dữ liệu ngẫu nhiên.

Cũng là $O(n \log n)$ trong mọi trường hợp và không cần bộ nhớ phụ (in-place), nhưng truy cập bộ nhớ nhảy cóc (cha-con cách xa nhau) nên tận dụng cache kém. Vì vậy dù cùng độ phức tạp, thực tế nó chậm hơn các thuật toán còn lại.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm

<https://github.com/Hoang-UIT07/sorting-benchmark>